

SECTION 4: WINDOW FUNCTION

Lab 1: Film ranking (The Basics).

Business Scenario: The Content Team wants to schedule a movie marathon. They need the top 3 longest films within each Category.

Task:

1. Join film, film_category, and category tables.
2. Use DENSE_RANK() to rank film lengths (descending) per category.
3. Hint: Use PARTITION BY category name.

```
Query Query History
1 WITH RankedFilms AS (
2     SELECT
3         c.name AS category_name,
4         f.title,
5         f.length,
6         DENSE_RANK() OVER (
7             PARTITION BY c.name
8             ORDER BY f.length DESC
9         ) AS rank_in_category
10    FROM film f
11   JOIN film_category fc ON f.film_id = fc.film_id
12   JOIN category c ON fc.category_id = c.category_id
13 )
14 SELECT
15     category_name,
16     title,
17     length,
18     rank_in_category
19 FROM RankedFilms
20 WHERE rank_in_category <= 3;
```

- Output: 64 rows

	category_name character varying (25)	title character varying (255)	length smallint	rank_in_category bigint
1	Action	Worst Banger	185	1
2	Action	Darn Forrester	185	1
3	Action	Casualties Encino	179	2
4	Action	Quest Mussolini	177	3
5	Animation	Pond Seattle	185	1
6	Animation	Gangs Pride	185	1
7	Animation	Theory Mermaid	184	2
8	Animation	Sons Interview	184	2
9	Animation	Lawless Vision	181	3
10	Children	Wrong Behavior	178	1
11	Children	Fury Murder	178	1
12	Children	Empire Malkovich	177	2
13	Children	Model Fish	175	3
14	Classics	Conspiracy Spirit	184	1
15	Classics	Dracula Crystal	176	2

Lab 2: Growth analysis (Time Series).

AI Context: For Sales Forecasting models, raw revenue is not enough. We need the Month-over-Month (MoM) Growth Rate as a feature.

Task:

1. Create a CTE to calculate total revenue per month (DATE_TRUNC).
2. Use LAG() to retrieve the previous month's revenue.
3. Calculate Growth %: $\frac{\text{Current} - \text{Previous}}{\text{Previous}} \cdot 100$

```

Query Query History
1 WITH MonthlyRevenue AS (
2     SELECT
3         DATE_TRUNC('month', payment_date) AS payment_month,
4         SUM(amount) AS current_revenue
5     FROM payment
6     GROUP BY DATE_TRUNC('month', payment_date)
7 )
8 SELECT
9     payment_month,
10    current_revenue,
11    LAG(current_revenue, 1) OVER (ORDER BY payment_month) AS previous_revenue,
12    ROUND(
13        (current_revenue - LAG(current_revenue, 1) OVER (ORDER BY payment_month))
14        / LAG(current_revenue, 1) OVER (ORDER BY payment_month) * 100
15    , 2) AS growth_rate_pct
16 FROM MonthlyRevenue;

```

- Output: 4 rows

Showing rows: 1 to 4   Page No: 1 of 1    				
	payment_month timestamp without time zone	current_revenue numeric	previous_revenue numeric	growth_rate_pct numeric
1	2007-02-01 00:00:00	8351.84	[null]	[null]
2	2007-03-01 00:00:00	23886.56	8351.84	186.00
3	2007-04-01 00:00:00	28559.46	23886.56	19.56
4	2007-05-01 00:00:00	514.18	28559.46	-98.20

Lab 3: Customer lifetime value (Running Total)

AI Context: We need to track the Customer Lifetime Value (CLV) at the exact moment of each transaction to predict Churn.

Task:

1. Use the payment table.
2. Calculate the Cumulative Sum of the amount column for each customer.

3. Order the data chronologically by payment_date.

Query		Query History	
1	SELECT		
2	customer_id,		
3	payment_date,		
4	amount,		
5	SUM(amount) OVER (		
6	PARTITION BY customer_id		
7	ORDER BY payment_date		
8	) AS customer_lifetime_value		
9	FROM payment;		

- Output: 1000 rows

Showing rows: 1 to 1000				Page No: 1	of 15				
	customer_id smallint	payment_date timestamp without time zone	amount numeric (5,2)	customer_lifetime_value numeric					
1	1	2007-02-14 23:22:38.996577	5.99	5.99					
2	1	2007-02-15 16:31:19.996577	0.99	6.98					
3	1	2007-02-15 19:37:12.996577	9.99	16.97					
4	1	2007-02-16 13:47:23.996577	4.99	21.96					
5	1	2007-02-18 07:10:14.996577	4.99	26.95					
6	1	2007-02-18 12:02:25.996577	0.99	27.94					
7	1	2007-02-21 04:53:11.996577	3.99	31.93					
8	1	2007-03-01 07:19:30.996577	4.99	36.92					
9	1	2007-03-02 14:05:18.996577	3.99	40.91					
10	1	2007-03-02 16:30:04.996577	0.99	41.90					
11	1	2007-03-17 11:06:20.996577	4.99	46.89					
12	1	2007-03-18 02:25:55.996577	0.99	47.88					
13	1	2007-03-19 08:23:42.996577	0.99	48.87					

Lab 4: The “Top-N” filter challenge

The Challenge: Find the single highest-spending customer (Top 1) for each Country.
Problem: You cannot use WHERE rank = 1 directly in the SELECT statement.

Instructions:

1. Step 1: Join 4 tables (customer, address, city, country) to aggregate total spend by Customer and Country.
2. Step2: Use a CTE to apply RANK() partitioned by Country.
3. Step3: Select from the CTE where rank = 1.

- Script SQL:

WITH RankedFilms AS (

SELECT

c.name AS category_name,

```

    f.title,
    f.length,
    DENSE_RANK() OVER (
        PARTITION BY c.name
        ORDER BY f.length DESC
    ) AS rank_in_category
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
)
SELECT
    category_name,
    title,
    length,
    rank_in_category
FROM RankedFilms
WHERE rank_in_category <= 3;

```

```

WITH MonthlyRevenue AS (
    SELECT
        DATE_TRUNC('month', payment_date) AS payment_month,
        SUM(amount) AS current_revenue
    FROM payment
    GROUP BY DATE_TRUNC('month', payment_date)
)
SELECT
    payment_month,
    current_revenue,

```

```

LAG(current_revenue, 1) OVER (ORDER BY payment_month) AS
previous_revenue,

ROUND(

    (current_revenue - LAG(current_revenue, 1) OVER (ORDER BY
payment_month))

    / LAG(current_revenue, 1) OVER (ORDER BY payment_month) * 100

, 2) AS growth_rate_pct
FROM MonthlyRevenue;

```

```

SELECT

    customer_id,

    payment_date,

    amount,

    SUM(amount) OVER (

        PARTITION BY customer_id

        ORDER BY payment_date

    ) AS customer_lifetime_value
FROM payment;

```

```

WITH RankedCustomers AS (

    SELECT

        co.country,

        c.customer_id,

        c.first_name,

        c.last_name,

        SUM(p.amount) AS total_spend,

        RANK() OVER (

            PARTITION BY co.country

            ORDER BY SUM(p.amount) DESC

```

```

    ) as spending_rank
FROM customer c
JOIN address a ON c.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY
    co.country,
    c.customer_id,
    c.first_name,
    c.last_name
)
SELECT
    country,
    customer_id,
    first_name,
    last_name,
    total_spend
FROM RankedCustomers
WHERE spending_rank = 1;
- Output: 108 rows

```

Showing rows: 1 to 108   Page No: 1 of 1    					
	country character varying (50) 	customer_id integer 	first_name character varying (45) 	last_name character varying (45) 	total_spend numeric 
1	Afghanistan	218	Vera	Mccoy	67.82
2	Algeria	176	June	Carroll	151.68
3	American Samoa	320	Anthony	Schwab	47.85
4	Angola	528	Claude	Herzog	93.80
5	Anguilla	381	Bobby	Boudreau	99.68
6	Argentina	560	Jordan	Archuleta	129.71
7	Armenia	41	Stephanie	Mitchell	118.75
8	Austria	266	Nora	Herrera	113.74
9	Azerbaijan	334	Raymond	Mcwhorter	108.75
10	Bahrain	590	Seth	Hannon	108.76
11	Bangladesh	21	Michelle	Clark	146.68
12	Belarus	144	Clara	Shaw	189.60
13	Bolivia	431	Joel	Francisco	92.78
14	Brazil	178	Marion	Snyder	194.61